

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа №7

«Шаблоны классов.»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 15

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

1. Определить шаблон класса-контейнера (см. лабораторную работу №6).
2. Реализовать конструкторы, деструктор, операции ввода-вывода, присваивания.
3. Перегрузить операции, указанные в варианте.
4. Инстанцировать шаблон для стандартных типов данных (int, float, double).
5. Написать тестирующую программу, иллюстрирующую выполнение операций для контейнера, содержащего элементы стандартных типов данных.
6. Реализовать пользовательский класс (см. лабораторную работу №3).
7. Перегрузить для пользовательского класса операции ввода-вывода.
8. Перегрузить операции необходимые для выполнения операций контейнерного класса.
9. Инстанцировать шаблон для пользовательского класса.
10. Написать тестирующую программу, иллюстрирующую выполнение операций для контейнера, содержащего элементы пользовательского класса.

Класс- контейнер СПИСОК с ключевыми значениями типа int.

Реализовать операции:

[] – доступа по индексу;

int() – определение размера списка;

* вектор – умножение элементов списков $a[i]*b[i]$;

Пользовательский класс `Pair` (пара чисел). Пара должна быть представлено двумя полями: типа `int` для первого числа и типа `double` для второго. Первое число при выводе на экран должно быть отделено от второго числа двоеточием.

АНАЛИЗ.

1. Согласно принципам инкапсуляции поля классов должны быть приватными, а описанные в постановке методы — публичными.

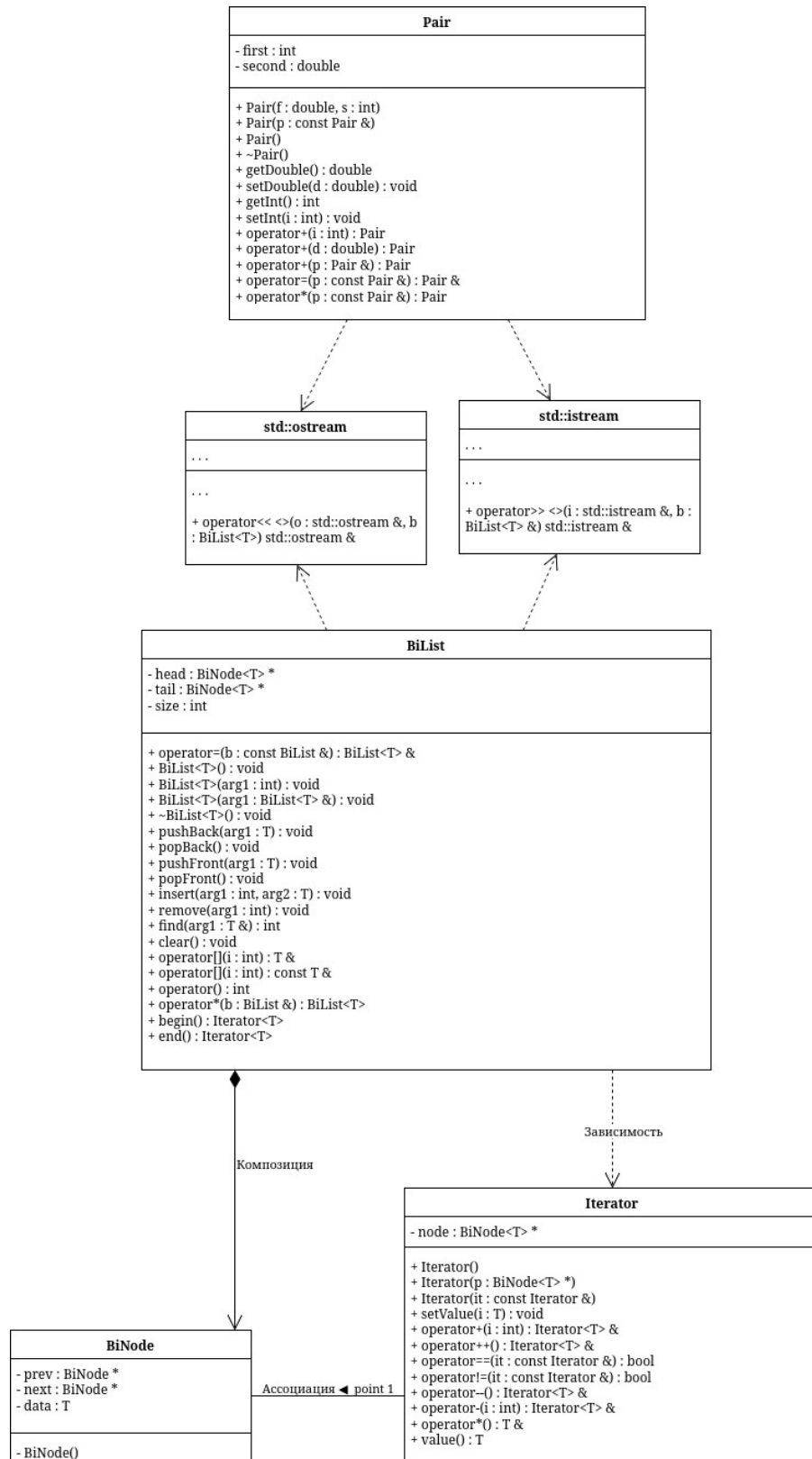
2. Для работы с приватными полями реализуем гетеры и сетеры.

3. Для объявления переменных заданного класса реализуем конструкторы: обычный (принимает на вход значения полей класса), копирования и «пустой конструктор», выставляющий значения по умолчанию.

4. Используем темплейты для универсализации листа для всех типов данных.

5. Перегрузим оператор умножения для пары, так как лист этого требует.

UML-ДИАГРАММА



КОД Pair.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
#include <ostream>

class Pair {
    int first;
    double second;

public:
    Pair(double, int);
    Pair(const Pair &);
    Pair();
    ~Pair();
    double getDouble() const;
    void setDouble(double);
    int getInt() const;
    void setInt(int);
    Pair operator+(Pair &p);
    Pair operator+(int);
    Pair operator+(double);
    Pair &operator=(const Pair &p);
    Pair operator*(const Pair& p);
    friend std::ostream &operator<<(std::ostream &, const Pair &);
    friend std::istream &operator>>(std::istream &, Pair &);
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
```

КОД Pair.cpp

```
#include "Pair.h"
#include <iostream>

Pair::Pair(double a, int b) {
    first = b;
    second = a;
}

Pair::Pair(const Pair &f) {
    second = f.second;
    first = f.first;
}

Pair::Pair() {
```

```

    second = 0;
    first = 0;
}

Pair::~Pair() {
    second = 0;
    first = 0;
}

double Pair::getDouble() const {
    return second;
}

void Pair::setDouble(double a) {
    second = a;
}

int Pair::getInt() const {
    return first;
}

void Pair::setInt(int a) {
    first = a;
}

Pair &Pair::operator=(Pair const &p) {
    second = p.second;
    first = p.first;
    return *this;
}

std::ostream &operator<<(std::ostream &stream, const Pair &p) {
    std::cout << p.first << " : " << p.second;
    return stream;
}

std::istream &operator>>(std::istream &stream, Pair &p) {
    std::cout << "Double? ";
    stream >> p.second;
    std::cout << "Int? ";
    stream >> p.first;
    return stream;
}

Pair Pair::operator+(Pair &p) {

```

```

    Pair second(*this);
    second.second += p.second;
    second.first += p.first;
    return second;
}

Pair Pair::operator+(int a) {
    Pair tmp(*this);
    tmp.first += a;
    return tmp;
}

Pair Pair::operator+(double a) {
    Pair tmp(*this);
    tmp.second += a;
    return tmp;
}

Pair Pair::operator*(const Pair& p) {
    double d = p.second * this->second;
    int i = p.first * this->first;
    return Pair(d, i);
}

```

KOД Student.h

```

#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_Student_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_Student_H
#include <string>
#include "Person.h"

class Student : public Person {
    std::string subject;
    int grade;

public:
    Student();

    Student(std::string name, int age, std::string subject, int grade);

    Student(Person, std::string subject, int grade);

```

```

Student(Student&);

~Student();

std::string getSubject();

void setSubject(std::string);

int getGrade();

void setGrade(int);

Student &operator=(Student &p);

friend std::ostream &operator<<(std::ostream &, Student &);

friend std::istream &operator>>(std::istream &, Student &);
};

```

```

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_Student_H

```

КОД bidirectional.h

```

#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_BI_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_BI_H
#include <iosfwd>

template<class T>
class BiList;
template<class T>
std::ostream &operator<<(std::ostream &, BiList<T>);
template<class T>
std::istream &operator>>(std::istream &, BiList<T> &);
template<class T>
class Iterator;

template<class T>
class BiNode {
    friend BiList<T>;
    friend Iterator<T>;
    BiNode *prev;
    BiNode *next;
    T data;
};

```



```

    BiNode() { prev = nullptr, next = nullptr, data = T(); }
};

template<class T>
class Iterator {
    BiNode<T> *node;

public:
    Iterator() { node = nullptr; }
    Iterator(BiNode<T> *p) { node = p; }
    Iterator(const Iterator &it) { node = it.node; }
    bool operator==(const Iterator &it) { return node == it.node; }
    bool operator!=(const Iterator &it) { return node != it.node; }

    Iterator<T> &operator++();
    Iterator<T> &operator--();
    Iterator<T> &operator+(int);
    Iterator<T> &operator-(int);

    T& operator*() { return node->data; }
    T value() { return node->data; }
    void setValue(T i) { node->data = i; }
};

template<class T>
class BiList {
    BiNode<T> *head;
    BiNode<T> *tail;
    int size;

public:
    BiList<T>();
    BiList<T>(int);
    BiList<T>(BiList<T> &);
    ~BiList<T>() { clear(); }

    void pushBack(T);
    void popBack();
    void pushFront(T);
    void popFront();
    void insert(int, T);
    void remove(int);
    int find(T &);
    void clear();

```

```

T &operator[](int);
const T &operator[](int) const;
BiList<T> &operator=(const BiList &);
int operator()() { return size; }
BiList<T> operator*(BiList &);

friend std::ostream &operator<< <>(std::ostream &, BiList<T>);
friend std::istream &operator>> <>(std::istream &, BiList<T> &);

Iterator<T> begin() { return Iterator(head); }
Iterator<T> end() { return Iterator(tail ? tail->next : nullptr); }
};
#include "bidirectional.cpp"
#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_BI_H

```

KOД bidirectional.cpp

```

#include "bidirectional.h"
#include <iostream>
#include <fstream>

template<class T>
void BiList<T>::pushBack(T data) {
    auto node = new BiNode<T>;
    node->data = data;
    if (size == 0) {
        head = node;
        tail = node;
        size++;
        return;
    }

    node->prev = tail;
    tail->next = node;
    tail = node;
    size++;
}

template<class T>
void BiList<T>::popBack() {
    if (size == 0)
        return;

    auto del = tail;

```

```

    tail = tail->prev;

    if (tail != nullptr)
        tail->next = nullptr;
    else
        head = nullptr;

    delete del;
    size--;
}

template<class T>
void BiList<T>::pushFront(T data) {
    auto node = new BiNode<T>;
    node->data = data;
    if (size == 0) {
        head = node;
        tail = node;
        size++;
        return;
    }

    node->next = head;
    head->prev = node;
    head = node;
    size++;
}

```

```

template<class T>
void BiList<T>::popFront() {
    if (size == 0)
        return;

    auto del = head;
    head = head->next;

    if (head != nullptr)
        head->prev = nullptr;
    else
        tail = nullptr;
}

```

```

delete del;
size--;

}

template<class T>
void BiList<T>::insert(int idx, T data) {
    if (idx == size) {
        pushBack(data);
        return;
    }
    if (idx == 0) {
        pushFront(data);
        return;
    }

    if (!(0 < idx && idx < size)) {
        return;
    }

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

    auto newNode = new BiNode<T>;
    newNode->data = data;

    node->prev->next = newNode;
    newNode->prev = node->prev;

    node->prev = newNode;
    newNode->next = node;

    size++;

}

template<class T>
void BiList<T>::remove(int idx) {
    if (!(0 <= idx && idx < size)) {

```

```

        return;
    }
    if (idx == 0) {
        popFront();
        return;
    }
    if (idx == size - 1) {
        popBack();
        return;
    }

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

    node->next->prev = node->prev;
    node->prev->next = node->next;

    delete node;
    size--;
}

```

```

template<class T>
int BiList<T>::find(T &data) {
    auto node = head;
    int i = 0;
    while (node != nullptr) {
        if (node->data == data) {
            return i;
        }
        i++;
        node = node->next;
    }

    return -1;
}

```

```

template<class T>
void BiList<T>::clear() {
    auto node = head;
    while (node != nullptr) {
        auto tmp = node->next;
        delete node;
    }
}

```

```

        node = tmp;
    }
    head = nullptr;
    tail = nullptr;
    size = 0;
}

```

```

template<class T>
T &BiList<T>::operator[](int idx) {
    if (!(0 <= idx && idx < size)) {
        exit(1);
    }

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

    return node->data;
}

```

```

template<class T>
const T &BiList<T>::operator[](int idx) const {
    if (!(0 <= idx && idx < size)) {
        exit(1);
    }

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

    return node->data;
}

```

```

template<class T>
BiList<T> &BiList<T>::operator=(const BiList<T> &l) {
    if (this == &l)
        return *this;

    this->clear();
    for (int i = 0; i < l.size; i++) {

```

```

        this->pushBack(l[i]);
    }

    return *this;
}

template<class T>
BiList<T> BiList<T>::operator*(BiList &l) {
    BiList res;
    int range = std::min(l.size, this->size);
    for (int i = 0; i < range; i++) {
        res.pushBack((*this)[i] * l[i]);
    }
    return res;
}

template<class T>
BiList<T>::BiList() {
    head = nullptr;
    tail = nullptr;
    size = 0;
}

template<class T>
BiList<T>::BiList(int s) {
    head = nullptr;
    tail = nullptr;
    size = 0;
    for (int i = 0; i < s; i++)
        pushBack(T());
}

template<class T>
BiList<T>::BiList(BiList<T> &l) {
    head = nullptr;
    tail = nullptr;
    size = 0;
    for (int i = 0; i < l.size; i++) {
        pushBack(l[i]);
    }
}

```

```

template<class T>
std::ostream &operator<<(std::ostream &stream, BiList<T> l) {
    for (int i = 0; i < l(); i++)
        stream << l[i] << '\t';
    return stream;
}

```

```

template<class T>
std::istream &operator>>(std::istream &stream, BiList<T> &l) {
    l.clear();
    std::cout << "Size? ";
    int s;
    stream >> s;
    std::cout << "Elements? ";
    for (int i = 0; i < s; i++) {
        T a;
        stream >> a;
        l.pushBack(a);
    }
    return stream;
}

```

```

template<class T>
Iterator<T> &Iterator<T>::operator+(int a) {
    for (int i = 0; i < a; i++)
        node = node->next;
    return *this;
}

```

```

template<class T>
Iterator<T> &Iterator<T>::operator-(int a) {
    for (int i = 0; i < a; i++)
        node = node->prev;
    return *this;
}

```

```

template<class T>
Iterator<T> &Iterator<T>::operator--() {
    node = node->prev;
    return *this;
}

```

```

template<class T>
Iterator<T> &Iterator<T>::operator++() {
    node = node->next;
}

```



```
    return *this;  
}
```

КОД main.cpp

```
#include <iostream>  
#include "Pair.h"  
#include "bidirectional.h"  
  
int main()  
{  
    BiList<Pair> l1, l2;  
    std::cin >> l1 >> l2;  
    std::cout << l1 << '\n' << l2 << '\n';  
    std::cout << "l1[0] + l2[1]\n";  
    std::cout << l1[0] + l2[1] << '\n';  
    std::cout << "Multiplication:\n" << l1 * l2;  
}
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Size? 3

Elements? Double? 1.1

Int? 1

Double? 2.2

Int? 2

Double? 3.3

Int? 3

Size? 2

Elements? Double? 4.4

Int? 4

Double? 5.5

Int? 5

1 : 1.1 2 : 2.2 3 : 3.3

4 : 4.4 5 : 5.5

$l1[0] + l2[1]$

6 : 6.6

Multiplication:

4 : 4.84 10 : 12.1

ВОПРОСЫ

1. В чем смысл использования шаблонов?

Шаблоны позволяют отделить алгоритм от конкретных типов данных и создавать универсальный код, который работает с различными типами без переписывания функций или классов.

2. Каковы синтаксис/семантика шаблонов функций?

Шаблон функции задается конструкцией `template <параметры_шаблона>` перед определением функции. Типы данных передаются как параметры шаблона, а компилятор автоматически создает конкретную версию функции при ее вызове с определенным типом.

3. Каковы синтаксис/семантика шаблонов классов?

Шаблон класса определяется через `template <параметры_шаблона>` перед объявлением класса. Класс становится параметризованным и может работать с любыми типами данных, переданными в качестве параметров. Конкретный класс создается компилятором при инстанцировании.

4. Что такое параметры шаблона функции?

Это параметры, задающие типы данных, которые будут использоваться в шаблонной функции вместо конкретных типов.

5. Перечислите основные свойства параметров шаблона функции.

Параметры шаблона используются как абстрактные типы, задаются в заголовке шаблона, подставляются компилятором автоматически при вызове функции и позволяют создавать разные версии функции для разных типов данных.

6. Как записывать параметр шаблона?

Параметр шаблона записывается в угловых скобках после слова `template`, например через ключевые слова `class` или `typename`.

7. Можно ли перегружать параметризованные функции?

Да, параметризованные функции можно перегружать.

8. Перечислите основные свойства параметризованных классов.

Параметризованные классы позволяют создавать семейство классов для разных типов данных, поддерживают обобщенное программирование, могут содержать статические элементы и дружественные функции, а также могут наследоваться от обычных и шаблонных классов.

9. Все ли компонентные функции параметризованного класса являются параметризованными?

Да, компонентные функции шаблонного класса работают с параметрами шаблона этого класса.

10. Являются ли дружественные функции, описанные в параметризованном классе, параметризованными?

Нет, дружественные функции не становятся автоматически параметризованными.

11. Могут ли шаблоны классов содержать виртуальные компонентные функции?

Да, шаблоны классов могут содержать виртуальные компонентные функции, но сами шаблоны функций виртуальными быть не могут.

12. Как определяются компонентные функции параметризованных классов вне определения шаблона класса?

Перед определением функции повторно указывается заголовок шаблона, а имя функции записывается с указанием шаблонного класса и его параметров.

13. Что такое инстанцирование шаблона?

Инстанцирование — это процесс генерации компилятором конкретного класса или функции по шаблону с подстановкой определенного типа данных.

14. На каком этапе происходит генерирование определения класса по шаблону?

Генерирование происходит на этапе компиляции, когда создается экземпляр шаблона с конкретным типом.